Clase 11 · Programa 04: servidor y cliente TCP de Internet

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · jueves 1 oct 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. El par de programas 04

Primer par en C con comunicación real por **sockets TCP en el puerto 4898**: el cliente pide la conexión y el servidor responde con texto. Dos terminales (misma máquina con **127.0.0.1** o máquinas distintas); si el puerto difiere, no conecta.

Cabecera	Para qué
`<string.h>`	`memset`, `memcpy`
`<sys/types.h>`, `<sys/socket.h>`	Funciones de sockets
`<netinet/in.h>`	`struct sockaddr_in` (IPv4)
`<netdb.h>`	DNS y `struct hostent`
`<unistd.h>`	`getpid`, `write`, `close`

El servidor declara dos `struct sockaddr_in` (**servidor** y **cliente**) y un `struct hostent *info_cliente`. `hostent` da la información del otro extremo.

2. char = byte y ASCII

- 1 byte = 8 bits = **256 valores** (0–255); para texto plano, 1 `char` = 1 byte.
- **ASCII básico (0–127):** del 0 al 32 son de control: 0 = NUL, 9 = tabulador, **10 = LF** (Line Feed), **13 = CR** (Carriage Return) — de las máquinas de escribir: bajar el papel y regresar el carro. Visibles desde el 33 (!).
- **ASCII extendido (128–255):** acentos, ñ y Ñ.
- Buffer `char buffer[128]`. Ojo: el Enter suma 1–2 bytes al mensaje.

3. El servidor paso a paso

1. `socket()` → descriptor **fds** (normalmente el 3).
2. Inicializar a mano: `memset(&servidor, 0, sizeof(servidor))`, `sin_family = AF_INET`, `sin_addr.s_addr = INADDR_ANY` (todas las interfaces), `sin_port = htons(4898)`.
3. `bind(fds, (struct sockaddr *)&servidor, sizeof(servidor));`
4. `listen(fds, 1);` (backlog 1).
5. `fdc = accept(fds, (struct sockaddr *)&cliente, &longitud_cliente);` — **bloqueante**.

6–7. (Opcional) `gethostbyaddr()` para imprimir IP o dominio del cliente.

bind() vs. accept() (examen)

`bind()` recibe la estructura **servidor**, inicializada por el programador, y su **tamaño**. `accept()` recibe la estructura **cliente**, que **llena él solo**, y un **apuntador** a la longitud. `fds` solo acepta conexiones; **todo el flujo de datos va por** `fdc`.

Punto de mejora: si `gethostbyaddr()` no resuelve devuelve NULL y `info_cliente->h_name` causa **fallo de segmentación** → validar `if (info_cliente != NULL)`.

4. El cliente paso a paso

Recibe `argv[1]` (IP o dominio: 127.0.0.1, 132.248.59.6) y `argv[2]` (puerto).

1. `socket()` → `fd`.
2. `memset`, `sin_family = AF_INET`, `sin_port = htons(atoi(argv[2]))`.
3. `struct hostent *he = gethostbyname(argv[1]);` y `memcpy(&cliente.sin_addr, he->h_addr, he->h_length);` (copia los 4 bytes de la IP).
4. `connect(fd, (struct sockaddr *)&servidor, sizeof(servidor));` — mismos 3 tipos de argumentos que `bind()` y al mismo nivel que `accept()`.

5. send(), recv() y write(1, …)

Mensaje del servidor: "El mensaje del servidor es : conexión exitosa!" (**46 bytes** en la prueba).

Llamada	Detalle
`send(fdc, buffer, strlen(buffer), 0);`	socket, buffer, longitud, flags (0 por defecto)
`int N = recv(fd, buffer, sizeof(buffer), 0);`	Misma firma; devuelve **N** = bytes realmente recibidos (46)
`write(1, buffer, N);`	Descriptor **1 = stdout** (0 teclado, 2 error); se pasa **N** para no imprimir basura

`send()` y `recv()` van **en pareja**; con `sizeof(buffer)` en `write()` se imprime basura.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Servidor: socket → (memset, AF_INET, INADDR_ANY, htons) → bind → listen → accept (bloqueante, devuelve fdc). **Cliente:** socket → gethostbyname + memcpy → connect.

> **Puntos de examen**
>
> bind() recibe servidor inicializado y su tamaño; accept() llena cliente y recibe &longitud. fds acepta, fdc comunica. Validar NULL de gethostbyaddr(). `write(1, buffer, N)` con N de recv().

Concepto	Para recordar
ASCII	0–127 básico (LF 10, CR 13), 128–255 extendido
send/recv	4 argumentos: socket, buffer, longitud, flags = 0
Descriptores	0 stdin, 1 stdout, 2 stderr